

### **Pregunta 1** (+0.5 cada una)

Indica qué resultado producen los siguientes programas en Java:

1.-

```
public class Ejercicio1 {
    public static void main(String args[]) {
        int x=10;
        int y=0;
        while (y<x) {
            x += y;
        }
        System.out.println(y);
    }
}
```

2.-

```
public class Ejercicio2 {
    public static void main(String args[]) {
        int x = 0;
        while (x++ <= 10) {
            System.out.print(x);
            if (x%4 == 0)
                System.out.println();
        }
    }
}
```

3.-

```
public class Ejercicio3 {
    public static void main(String[] args) {
        int suma = 0;
        for (int i = 0; i < 4; i++) {
            for (int j = 3; j > 0; j--) {
                suma = i * 10 + j;
            }
            System.out.println(suma);
        }
    }
}
```

4.-

```
public class Ejercicio4 {
    public static void main(String args[]) {
        int TAM = 5;
        int array[] = new int[TAM];
        array[0] = 2;
        for (int i = 1; i < TAM; i++) {
            array[i] = i * array[i - 1];
        }
        for (int i = 0; i < TAM; i++) {
            System.out.print(array[i] + " ");
        }
    }
}
```

5.-

```
public class Ejercicio5 {  
    public static void main(String args[]) {  
        int TAM = 10;  
        int array[] = new int[TAM];  
        for (int i = 0; i < TAM; i++) {  
            if (i%3==0)  
                array[i] += 5;  
            else if (i%2==0)  
                array[i] += 7;  
        }  
        for (int i = 0; i < TAM; i++) {  
            System.out.print(array[i] + " ");  
        }  
    }  
}
```

## Pregunta 2

Realiza un programa en java que reciba frases y de cada frase escriba la palabra más larga y el número de caracteres que tiene.

El programa acaba cuando la frase introducida sea "fin", tanto si se ha escrito con mayúsculas o minúsculas. **(+0.25 puntos)**

Para la lectura de la frase tendrás que usar un método con la siguiente cabecera:

```
public static String leerFrase(Scanner sc) (+0.5 puntos)
```

Mediante este método se le pide al usuario que introduzca una frase que el método devolverá a quien lo invoque. Si el usuario introduce una frase vacía, el método no la devuelve, sino que volverá a pedir una nueva frase.

Una vez obtenida cada frase, en el programa principal se llamará a un método que reciba la frase y escriba dentro del propio método la palabra de mayor longitud, así como su longitud. En este caso, la cabecera será:

```
public static void palabraMasLarga(String frase) (+1.25 puntos)
```

Consideramos que las distintas palabras de una frase están separadas entre sí por uno o varios espacios en blanco exclusivamente. Si hay varias con la mayor longitud, mostramos la primera.

### **Ejemplo de ejecución:**

Introduzca una frase:

Frase primera de prueba

La palabra de mayor longitud es: primera

Su longitud es: 7

Introduzca una frase:

gracias por todo

La palabra de mayor longitud es: gracias

Su longitud es: 7

Introduzca una frase:

FIN

### **Pregunta 3**

2.- Dos números son **gemelos** si son primos (el número uno no se considera primo) y además se diferencian en dos unidades.

Ejemplos de parejas de números gemelos:

3 y 5  
5 y 7  
17 y 19  
29 y 31  
101 y 103

Se pide desarrollar un programa que pida parejas de números y determine si son gemelos. Para ello usa los siguientes métodos en el programa principal:

- Implementa un método que siga la siguiente cabecera para determinar si un número es primo (devuelve true si lo es y false si no):

**public static boolean esPrimo(int num); (+0.5 puntos)**

- Implementa un método que siga la siguiente cabecera para determinar si 2 números son gemelos (devuelve true si lo son y false si no lo son), haciendo uso del método anterior:

**public static boolean sonGemelos(int num1, int num2); (+1 punto)**

En el programa principal se pedirán parejas de números que pasaremos como parámetros al método **sonGemelos**, indicando para cada pareja si son gemelos o no. El usuario indicará de alguna manera que no desea introducir ninguno más (se deja a tu elección la condición para parar de pedir números). **(+0.5 puntos)**

**Ejemplo** de ejecución (en este caso para cuando los dos números son cero):

Introduzca un número entero positivo: 4  
Introduzca otro número entero positivo: 6  
No son gemelos  
Introduzca un número entero positivo: 101  
Introduzca otro número entero positivo: 103  
Son gemelos  
Introduzca un número entero positivo: 0  
Introduzca otro número entero positivo: 0

#### Pregunta 4 (+0.5 cada una)

Indica qué se mostrará por pantalla cuando se ejecute cada una de estas clases:

```
class Uno{
    private static int metodo(){
        int valor=0;
        try{
            valor = valor +1;
            valor = valor + Integer.parseInt("42");
            valor = valor + 1;
            System.out.println("Valor al final del try: " + valor);
        }catch(NumberFormatException e){
            valor = valor + Integer.parseInt("42");
            System.out.println("Valor al final del catch: " + valor);
        }finally{
            valor = valor + 1;
            System.out.println("Valor al final de finally: " + valor);
        }
        valor = valor + 1;
        System.out.println("Valor antes del return: " + valor);
        return valor;
    }

    public static void main(String[] args){
        try{
            System.out.println(metodo());
        }catch(Exception e){
            System.err.println("Excepcion en metodo()");
            e.printStackTrace();
        }
    }
}
```

```
class Dos{
    private static int metodo(){
        int valor=0;
        try{
            valor = valor +1;
            valor = valor + Integer.parseInt("W");
            valor = valor + 1;
            System.out.println("Valor al final del try: " + valor);
        }catch(NumberFormatException e){
            valor = valor + Integer.parseInt("42");
            System.out.println("Valor al final del catch: " + valor);
        }finally{
            valor = valor + 1;
            System.out.println("Valor al final de finally: " + valor);
        }
        valor = valor + 1;
        System.out.println("Valor antes del return: " + valor);
        return valor;
    }

    public static void main(String[] args){
        try{
            System.out.println(metodo());
        }catch(Exception e){
            System.err.println("Excepcion en metodo()");
            e.printStackTrace();
        }
    }
}
```

```

class Tres{
    private static int metodo(){
        int valor=0;
        try{
            valor = valor + 1;
            valor = valor + Integer.parseInt("W");
            valor = valor + 1;
            System.out.println("Valor al final del try: " + valor);
        }catch(NumberFormatException e){
            valor = valor + Integer.parseInt("W");
            System.out.println("Valor al final del catch: " + valor);
        }finally{
            valor = valor + 1;
            System.out.println("Valor al final de finally: " + valor);
        }
        valor = valor + 1;
        System.out.println("Valor antes del return: " + valor);
        return valor;
    }

    public static void main(String[] args){
        try{
            System.out.println(metodo());
        }catch(Exception e){
            System.err.println("Excepcion en metodo()");
            e.printStackTrace();
        }
    }
}

```

```

import java.io.*;

class Cuatro{
    private static int metodo(){
        int valor=0;
        try{
            valor = valor + 1;
            valor = valor + Integer.parseInt("W");
            valor = valor + 1;
            System.out.println("Valor al final del try: " + valor);
            throw new IOException();
        }catch(IOException e){
            valor = valor + Integer.parseInt("42");
            System.out.println("Valor al final del catch: " + valor);
        }finally{
            valor = valor + 1;
            System.out.println("Valor al final de finally: " + valor);
        }
        valor = valor + 1;
        System.out.println("Valor antes del return: " + valor);
        return valor;
    }

    public static void main(String[] args){
        try{
            System.out.println(metodo());
        }catch(Exception e){
            System.err.println("Excepcion en metodo()");
            e.printStackTrace();
        }
    }
}

```

### **Pregunta 5 (+0.3 cada apartado)**

Create two methods that deal with exceptions. One of the methods is the `main()` method, which will call another method. If an exception is thrown in the other method, `main()` must deal with it. A finally statement will be included to indicate that the program has completed. The method that `main()` will call will be named **reverse**, and it will reverse the order of the characters in a String. If the String contains no characters, reverse will propagate an exception up to the `main()` method.

1. Create a class called **Propagate** and a `main()` method, which will remain empty for now.
2. Create a method called **reverse**. It takes an argument of a String and returns a String.
3. In **reverse**, check whether the String has a length of 0 by using the `String.length()` method. If the length is 0, the reverse method will throw an exception.
4. Now include the code to reverse the order of the String.
5. Now in the `main()` method you will attempt to call this method and deal with any potential exceptions. Additionally, you will include a finally statement that displays when `main()` has finished.